INTRODUCTION TO LOGIC I

JOONA PIIRAINEN

Abstract. Notes for the University of Helsinki course MAT21014, taught in Finnish from 03.09.2026–17.10.2026.

Contents

1. Course information 1
2. Lecture notes 1
2.1. 03.09.2026 1
2.2. 07.09.2026 2
2.3. 10.09.2026 5
2.4. 14.09.2026 7
2.5. 21.09.2026 8
2.6. 28.09.2026 11
2.7. 01.10.2026 14
2.8. 05.10.2026 17
2.9. 08.10.2026 19
2.10. 12.10.2026 23

1. Course information

The official course information is available on the University of Helsinki course page.

2. Lecture notes

2.1. **03.09.2026.**

2.1.1. *Propositional formulas.*

Definition 2.1. The set of *propositional formulas* is defined inductively as follows:

(1) Every propositional symbol p_n, where $n \in \mathbb{N}$, is a propositional formula.
(2) If A and B are propositional formulas, then
$$\neg A, \qquad (A \wedge B), \qquad (A \vee B), \qquad (A \to B), \qquad (A \leftrightarrow B)$$
are propositional formulas.
(3) No other strings are propositional formulas.

2.1.2. *Subformulas.*

Definition 2.2 (Immediate subformulas)**.** The immediate subformulas of a propositional formula are defined by its outermost connective:

(1) A propositional symbol p_n has no immediate subformulas.
(2) The only immediate subformula of $\neg A$ is A.

Date: Autumn 2026.

(3) The immediate subformulas of $(A \circ B)$ are A and B, where

$$\circ \in \{\wedge, \vee, \to, \leftrightarrow\}.$$

Definition 2.3 (Subformulas)**.** A formula B is a *subformula* of a formula A if $B = A$, or if there are formulas

$$A = A_0, A_1, \ldots, A_k = B$$

such that $k \geq 1$ and A_{i+1} is an immediate subformula of A_i for every $0 \leq i < k$.

2.1.3. *Structural induction.*

Theorem 2.4 (Structural induction)**.** *Let $P(A)$ be a statement about propositional formulas. Suppose that*

(1) $P(p_n)$ holds for every $n \in \mathbb{N}$;
(2) if $P(A)$ holds, then $P(\neg A)$ holds; and
(3) if $P(A)$ and $P(B)$ hold, then $P((A \circ B))$ holds for every

$$\circ \in \{\wedge, \vee, \to, \leftrightarrow\}.$$

Then $P(A)$ holds for every propositional formula A.

Example 2.5 (Structural induction)**.** For a formula A, let $v(A)$ be the number of occurrences of propositional symbols in A, and let $b(A)$ be the number of occurrences of binary connectives in A. We prove that

$$v(A) = b(A) + 1$$

for every propositional formula A.

If $A = p_n$, then $v(A) = 1$ and $b(A) = 0$, so the claim holds. Suppose next that the claim holds for a formula A. Negation introduces neither a propositional symbol nor a binary connective, and hence

$$v(\neg A) = v(A) = b(A) + 1 = b(\neg A) + 1.$$

Finally, suppose that the claim holds for formulas A and B, and let $\circ$ be any binary connective. Then

$$\begin{aligned} v((A \circ B)) &= v(A) + v(B) \\ &= (b(A) + 1) + (b(B) + 1) \\ &= b((A \circ B)) + 1. \end{aligned}$$

The result follows by structural induction. □

2.2. **07.09.2026.**

2.2.1. *Structural induction.*

Example 2.6 (Matching parentheses)**.** For a propositional formula A, let $l(A)$ and $r(A)$ denote the numbers of occurrences of left and right parentheses in A, respectively. We prove by structural induction that

$$l(A) = r(A)$$

for every propositional formula A.

If $A = p_n$, then $l(A) = 0 = r(A)$. Suppose next that the claim holds for a formula A. Negation introduces no parentheses, so

$$l(\neg A) = l(A) = r(A) = r(\neg A).$$

Finally, suppose that the claim holds for formulas A and B, and let $\circ$ be any binary connective. The formula $(A \circ B)$ introduces one left and one right parenthesis. Hence

$$\begin{aligned} l((A \circ B)) &= l(A) + l(B) + 1 \\ &= r(A) + r(B) + 1 \\ &= r((A \circ B)). \end{aligned}$$

The result follows by structural induction. □

2.2.2. *Closures.*

Definition 2.7 (Closure)**.** Let X be a set, let $B \subseteq X$, and, for $1 \leq i \leq k$, let

$$f_i \colon X^{n_i} \longrightarrow X$$

be a function. A subset $Y \subseteq X$ is *closed under* f_i if

$$f_i(y_1, \ldots, y_{n_i}) \in Y$$

whenever $y_1, \ldots, y_{n_i} \in Y$. The *closure of* B *under* $f_1, \ldots, f_k$, denoted by

$$\mathrm{cl}(B; f_1, \ldots, f_k),$$

is the smallest subset of X that contains B and is closed under every f_i.

Equivalently, the closure can be written as the intersection

$$\mathrm{cl}(B; f_1, \ldots, f_k) = \bigcap \{Y \subseteq X : B \subseteq Y \text{ and } Y \text{ is closed under every } f_i\}.$$

Example 2.8. Let $X = \mathbb{Z}$, $B = \{1\}$, and define functions $f, g \colon \mathbb{Z} \to \mathbb{Z}$ by

$$f(x) = x + 1 \qquad \text{and} \qquad g(x) = -x.$$

Then

$$\mathrm{cl}(\{1\}; f, g) = \mathbb{Z}.$$

Indeed, every subset of $\mathbb{Z}$ that contains 1 and is closed under g must also contain -1. Closure under f then gives 0, and repeated applications of f give every positive integer. Applying g to these integers gives every negative integer. Thus any such closed set is all of $\mathbb{Z}$, while $\mathbb{Z}$ itself clearly contains 1 and is closed under both functions.

Corollary 2.9 (Induction on a closure)**.** *Set*

$$C = \mathrm{cl}(B; f_1, \ldots, f_k).$$

To prove that every element of C satisfies a property P, it suffices to prove that

(1) every element of B satisfies P; and

(2) for each i, if $x_1, \ldots, x_{n_i}$ satisfy P, then $f_i(x_1, \ldots, x_{n_i})$ also satisfies P.

Proof. Let Y be the set of all elements of $\mathrm{cl}(B; f_1, \ldots, f_k)$ that satisfy P. The two assumptions imply that $B \subseteq Y$ and that Y is closed under every f_i. Since the closure is the smallest such set,

$$\mathrm{cl}(B; f_1, \ldots, f_k) \subseteq Y.$$

Hence every element of the closure satisfies P. □

Let Σ be the alphabet of propositional logic, let Σ^* be the set of all finite strings over Σ, and let

$$V = \{p_n : n \in \mathbb{N}\}$$

be the set of propositional symbols. The logical connectives determine the following functions on strings:

$$f_\neg(A) = \neg A$$

and

$$f_\circ(A, B) = (A \circ B), \qquad \circ \in \{\wedge, \vee, \to, \leftrightarrow\}.$$

The set of propositional formulas is precisely

$$\mathrm{cl}(V; f_\neg, f_\wedge, f_\vee, f_\to, f_\leftrightarrow).$$

In particular, if $S \subseteq \Sigma^*$ contains every propositional symbol and is closed under all functions associated with the logical connectives, then S contains every propositional formula.

2.2.3. *Semantics of propositional logic.*

Definition 2.10 (Truth assignment)**.** A *truth assignment* is a function

$$v\colon V \longrightarrow \{0,1\},$$

where V is the set of propositional symbols. The values 0 and 1 represent **false** and **true**, respectively.

The truth values of the logical connectives are defined as follows.

Definition 2.11 (Negation)**.** The negation $\neg A$ is true exactly when A is false.

A	$\neg A$
0	1
1	0

Definition 2.12 (Conjunction)**.** The conjunction $(A \wedge B)$ is true exactly when both A and B are true.

A	B	$(A \wedge B)$
0	0	0
0	1	0
1	0	0
1	1	1

Definition 2.13 (Disjunction)**.** The disjunction $(A \vee B)$ is true exactly when at least one of A and B is true.

A	B	$(A \vee B)$
0	0	0
0	1	1
1	0	1
1	1	1

Definition 2.14 (Implication)**.** The implication $(A \to B)$ is false exactly when A is true and B is false. It is important to note that implication is **not** symmetric.

A	B	$(A \to B)$
0	0	1
0	1	1
1	0	0
1	1	1

Definition 2.15 (Equivalence)**.** The equivalence $(A \leftrightarrow B)$ is true exactly when A and B have the same truth value.

A	B	$(A \leftrightarrow B)$
0	0	1
0	1	0
1	0	0
1	1	1

Definition 2.16 (Truth value)**.** Let A be a propositional formula and let v be a truth assignment. The *truth value* $v(A)$ of A is defined by the following rules:

(1) If A is a propositional symbol p_n, then $v(A)$ is already defined.

(2) If $A = \neg B$ and $v(B)$ is defined, then

$$v(A) = \begin{cases} 0, & \text{if } v(B) = 1, \\ 1, & \text{if } v(B) = 0. \end{cases}$$

Another way to write this is $v(A) = 1 - v(B)$.

(3) If $A = (B \wedge C)$ and $v(B)$ and $v(C)$ are defined, then $v(A) = 1$ exactly when $v(B) = v(C) = 1$, and $v(A) = 0$ otherwise. Another way to write this is $v(A)v(B)$.

(4) If $A = (B \vee C)$ and $v(B)$ and $v(C)$ are defined, then $v(A) = 1$ when at least one of $v(B) = 1$ and $v(C) = 1$ holds, and $v(A) = 0$ when $v(B) = v(C) = 0$.

(5) If $A = (B \to C)$ and $v(B)$ and $v(C)$ are defined, then $v(A) = 0$ when $v(B) = 1$ and $v(C) = 0$, and $v(A) = 1$ in all other cases.

(6) If $A = (B \leftrightarrow C)$ and $v(B)$ and $v(C)$ are defined, then $v(A) = 1$ iff $v(B) = v(C)$.

Definition 2.17 (Classification of propositional formulas)**.** A propositional formula A is

(1) a *tautology* if $v(A) = 1$ for every truth assignment v;

(2) a *contradiction* if $v(A) = 0$ for every truth assignment v;

(3) *contingent* if there are truth assignments v and w such that $v(A) = 1$ and $w(A) = 0$;

(4) *satisfiable* if $v(A) = 1$ for at least one truth assignment v; and

(5) *falsifiable* if $v(A) = 0$ for at least one truth assignment v.

Example 2.18 (De Morgan's laws)**.** Consider the formulas

$$(\neg(A \wedge B) \leftrightarrow (\neg A \vee \neg B)) \qquad \text{and} \qquad (\neg(A \vee B) \leftrightarrow (\neg A \wedge \neg B)).$$

Their truth values can be compared using the following table:

A	B	$\neg(A \wedge B)$	$(\neg A \vee \neg B)$	$\neg(A \vee B)$	$(\neg A \wedge \neg B)$
0	0	1	1	1	1
0	1	1	1	0	0
1	0	1	1	0	0
1	1	0	0	0	0

The two sides of each equivalence have the same truth value under every truth assignment. Consequently, both equivalences are tautologies. These equivalences are known as *De Morgan's laws*.

2.3. **10.09.2026.**

2.3.1. *Logical implication and equivalence.*

Definition 2.19 (Logical implication)**.** A formula A *logically implies* a formula B, written

$$A \Rightarrow B,$$

if, for every truth assignment v, $v(A) = 1$ implies $v(B) = 1$. Equivalently, $A \Rightarrow B$ exactly when $(A \to B)$ is a tautology.

Definition 2.20 (Logical equivalence)**.** Formulas A and B are *logically equivalent*, written

$$A \Leftrightarrow B,$$

if $v(A) = v(B)$ for every truth assignment v. Equivalently, $A \Leftrightarrow B$ exactly when $(A \leftrightarrow B)$ is a tautology.

The symbols $\Rightarrow$ and $\Leftrightarrow$ are used in the metalanguage to state relations between formulas; unlike $\to$ and $\leftrightarrow$, they are not connectives used to construct propositional formulas.

Definition 2.21 (Literal)**.** A *literal* is either a propositional symbol p_n or its negation $\neg p_n$. The former is a *positive literal*, and the latter is a *negative literal*.

Definition 2.22 (Disjunctive normal form)**.** A formula is in *disjunctive normal form* (DNF) if it is a disjunction of conjunctions of literals. In other words, it has the form

$$\bigvee_{i=1}^{m} \left(\bigwedge_{j=1}^{k_i} L_{ij} \right),$$

where $m \geq 1$, each $k_i \geq 1$, and every L_{ij} is a literal.

Definition 2.23 (Conjunctive normal form)**.** A formula is in *conjunctive normal form* (CNF) if it is a conjunction of disjunctions of literals. In other words, it has the form

$$\bigwedge_{i=1}^{m} \left(\bigvee_{j=1}^{k_i} L_{ij} \right),$$

where $m \geq 1$, each $k_i \geq 1$, and every L_{ij} is a literal.

For a literal L, let $\overline{L}$ denote its *complementary literal*, defined by

$$\overline{p_n} = \neg p_n \qquad \text{and} \qquad \overline{\neg p_n} = p_n.$$

De Morgan's laws show that DNF and CNF are dual: negating a DNF formula gives an equivalent CNF formula, and negating a CNF formula gives an equivalent DNF formula. More precisely,

$$\neg \bigvee_{i=1}^{m} \left(\bigwedge_{j=1}^{k_i} L_{ij} \right) \Leftrightarrow \bigwedge_{i=1}^{m} \left(\bigvee_{j=1}^{k_i} \overline{L_{ij}} \right),$$

and the analogous equivalence holds with $\wedge$ and $\vee$ interchanged.

Definition 2.24 (Boolean function)**.** An n-ary *truth function* is a function

$$f \colon \{0,1\}^n \longrightarrow \{0,1\}.$$

A propositional formula A whose propositional symbols are among $p_1, \ldots, p_n$ determines the truth function

$$f_A(x_1, \ldots, x_n) = v(A),$$

where $v(p_i) = x_i$ for every $1 \leq i \leq n$.

Theorem 2.25 (Normal form theorem)**.** *Let f be an n-ary truth function, where $n \geq 1$. Then there are formulas A_D in DNF and A_C in CNF, containing only the propositional symbols $p_1, \ldots, p_n$, such that*

$$f_{A_D} = f_{A_C} = f.$$

Consequently, every propositional formula is logically equivalent to a formula in DNF and to a formula in CNF.

Proof. We construct the formulas directly from the truth table of f.

To construct A_D, consider each row $a = (a_1, \ldots, a_n) \in \{0,1\}^n$ on which $f(a) = 1$, and define the formula

$$M_a = L_1^a \wedge \cdots \wedge L_n^a, \qquad L_i^a = \begin{cases} p_i, & \text{if } a_i = 1, \\ \neg p_i, & \text{if } a_i = 0. \end{cases}$$

Thus M_a is true under a truth assignment v exactly when $(v(p_1), \ldots, v(p_n)) = a$.

If there is at least one such row, let

$$A_D = \bigvee_{\substack{a \in \{0,1\}^n \\ f(a)=1}} M_a.$$

Under an assignment corresponding to $b \in \{0,1\}^n$, the formula A_D is true exactly when the disjunction contains M_b, which happens exactly when $f(b) = 1$. If f is constantly false, take $A_D = (p_1 \wedge \neg p_1)$. Thus $f_{A_D} = f$.

Dually, to construct A_C, consider each row a on which $f(a) = 0$ and define

$$C_a = K_1^a \vee \cdots \vee K_n^a, \qquad K_i^a = \begin{cases} p_i, & \text{if } a_i = 0, \\ \neg p_i, & \text{if } a_i = 1. \end{cases}$$

The clause C_a is false exactly on the row a. If there is at least one such row, let

$$A_C = \bigwedge_{\substack{a \in \{0,1\}^n \\ f(a)=0}} C_a.$$

This conjunction is true on a row b exactly when $f(b) = 1$. If f is constantly true, take $A_C = (p_1 \vee \neg p_1)$. Thus $f_{A_C} = f$.

All finite conjunctions and disjunctions above may be given any fixed binary parenthesization. Finally, for any propositional formula, list its distinct propositional symbols as $q_1, \ldots, q_n$ and apply the same construction with q_i in place of p_i. This gives logically equivalent formulas in DNF and CNF. □

Example 2.26 (Constructing normal forms from a truth function)**.** Consider the binary truth function whose truth table is

x_1	x_2	$f(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	0

The rows where $f = 1$ are $(0, 1)$ and $(1, 0)$. Replace a 1 in a row by p_i, replace a 0 by $\neg p_i$, and join the literals in each row with $\wedge$. This gives the formulas

$$M_{(0,1)} = (\neg p_1 \wedge p_2) \qquad \text{and} \qquad M_{(1,0)} = (p_1 \wedge \neg p_2).$$

Finally, join the formulas for the selected rows with $\vee$:

$$A = ((\neg p_1 \wedge p_2) \vee (p_1 \wedge \neg p_2)).$$

This is a DNF formula with $f_A = f$.

To construct a CNF formula, use instead the rows $(0, 0)$ and $(1, 1)$, where $f = 0$. The clauses that are false on exactly these rows are

$$C_{(0,0)} = (p_1 \vee p_2) \qquad \text{and} \qquad C_{(1,1)} = (\neg p_1 \vee \neg p_2).$$

Their conjunction

$$B = ((p_1 \vee p_2) \wedge (\neg p_1 \vee \neg p_2))$$

is in CNF and satisfies $f_B = f$. Thus $A \Leftrightarrow B$.

2.4. **14.09.2026.**

2.4.1. *Universal sets of connectives.*

Definition 2.27 (Universal set of connectives)**.** A set C of logical connectives is *universal*, or *functionally complete*, if, for every $n \geq 1$ and every truth function

$$f \colon \{0, 1\}^n \longrightarrow \{0, 1\},$$

there is a formula A, containing only the propositional symbols $p_1, \ldots, p_n$ and connectives from C, such that $f_A = f$.

Theorem 2.28. *The set $\{\neg, \wedge\}$ is universal.*

Proof. Let $f\colon \{0,1\}^n \to \{0,1\}$ be a truth function. The DNF construction gives a DNF formula A such that $f_A = f$. We prove by structural induction that every formula built from literals using $\wedge$ and $\vee$ is logically equivalent to a formula containing only $\neg$ and $\wedge$.

A literal is either p_i or $\neg p_i$, so the claim holds in the base case. Suppose that the induction hypothesis gives formulas B^* and C^*, containing only $\neg$ and $\wedge$, such that $B^* \Leftrightarrow B$ and $C^* \Leftrightarrow C$. Then

$$(B \wedge C) \Leftrightarrow (B^* \wedge C^*)$$

and, by De Morgan's law,

$$(B \vee C) \Leftrightarrow \neg(\neg B^* \wedge \neg C^*).$$

Both formulas on the right use only $\neg$ and $\wedge$. Structural induction therefore gives a formula A^*, containing only these connectives, such that $A^* \Leftrightarrow A$. Hence $f_{A^*} = f_A = f$, so $\{\neg, \wedge\}$ is universal. □

Theorem 2.29. *The set $\{\wedge, \vee\}$ is not universal.*

Proof. We prove by structural induction that every formula A containing only $\wedge$ and $\vee$ is false under the truth assignment v for which $v(p_i) = 0$ for every propositional symbol p_i.

If $A = p_i$, then $v(A) = 0$. Suppose that the claim holds for formulas B and C. Then

$$v(B \wedge C) = 0 \qquad \text{and} \qquad v(B \vee C) = 0,$$

so the claim also holds for formulas formed using either allowed connective. It therefore holds for every formula containing only $\wedge$ and $\vee$.

The unary truth function $f(x) = 1 - x$ has value $f(0) = 1$, so no such formula can represent it. Hence $\{\wedge, \vee\}$ is not universal. □

2.5. **21.09.2026.**

2.5.1. *Clauses and clause sets.*

Definition 2.30 (Clause)**.** A *clause* is a finite set of literals. It represents the disjunction of its members. For a truth assignment v, define

$$v(C) = 1 \quad \text{if and only if} \quad v(q) = 1 \text{ for some } q \in C,$$

and set $v(C) = 0$ otherwise. In particular, the *empty clause* $\emptyset$ is false under every truth assignment.

Definition 2.31 (Clause set)**.** A *clause set* $\mathcal{C}$ is a set of clauses. Define

$$v(\mathcal{C}) = 1 \quad \text{if and only if} \quad v(C) = 1 \text{ for every } C \in \mathcal{C},$$

and set $v(\mathcal{C}) = 0$ otherwise. Thus a finite clause set represents the conjunction of its clauses. It is *satisfiable* if some truth assignment makes it true, and *unsatisfiable* otherwise.

Remark 2.32. The empty clause and the empty clause set have different meanings: the empty clause is always false, whereas the empty clause set is always true (the condition on all its members holds vacuously). In particular, $\{\emptyset\}$ is an unsatisfiable clause set. A clause containing both p_i and $\neg p_i$ is always true.

Example 2.33 (Encoding CNF as a clause set)**.** Since $(p_0 \to p_1) \Leftrightarrow (\neg p_0 \vee p_1)$, the formula

$$p_0 \wedge (p_0 \to p_1)$$

is represented by $\mathcal{C} = \{\{p_0\}, \{\neg p_0, p_1\}\}$. The literals within a clause are joined by disjunction, and the clauses within a clause set are joined by conjunction. Order and repetitions do not matter in these set representations.

2.5.2. *The resolution rule and resolution proofs.*

Definition 2.34 (Resolution rule)**.** The *resolution rule* is

$$\frac{C_1 \cup \{p_i\} \qquad C_2 \cup \{\neg p_i\}}{C_1 \cup C_2}.$$

The conclusion is called a *resolvent*, and p_i is the *pivot*. In the usual application to clauses D, E with $p_i \in D$ and $\neg p_i \in E$, take $C_1 = D \setminus \{p_i\}$ and $C_2 = E \setminus \{\neg p_i\}$. Only the selected complementary pair is removed.

Example 2.35 (Resolution steps)**.** Modus ponens is the resolution step

$$\frac{\{p_0\} \qquad \{\neg p_0, p_1\}}{\{p_1\}}.$$

From the same two clauses one may obtain different resolvents by choosing different pivots. For example,

$$\frac{\{p_0, p_1\} \qquad \{\neg p_0, \neg p_1\}}{\{p_0, \neg p_0\}} \quad \text{or} \quad \frac{\{p_0, p_1\} \qquad \{\neg p_0, \neg p_1\}}{\{p_1, \neg p_1\}}.$$

Both conclusions are tautological clauses. Removing both complementary pairs at once to obtain $\emptyset$ would be invalid: the premises are simultaneously true when $v(p_0) = 1$ and $v(p_1) = 0$.

Definition 2.36 (Resolution proof)**.** A *resolution proof* of a clause C from a clause set $\mathcal{C}$ is a finite sequence $C_0, \ldots, C_m$, where $m \geq 0$, such that $C_m = C$ and, for every $i \leq m$, either

(1) $C_i \in \mathcal{C}$, or
(2) C_i follows by resolution from C_k, C_l for some $k, l < i$.

We write $\mathcal{C} \vdash_R C$ if such a proof exists. A proof of $\emptyset$ is called a *resolution refutation*.

2.5.3. *Soundness of resolution.*

Lemma 2.37 (Soundness of the resolution rule)**.** *If $v(C_1 \cup \{p_i\}) = v(C_2 \cup \{\neg p_i\}) = 1$, then $v(C_1 \cup C_2) = 1$.*

Proof. If $v(p_i) = 1$, then $v(\neg p_i) = 0$. Since the second premise is true, some literal of C_2 must be true. Hence $v(C_1 \cup C_2) = 1$. If $v(p_i) = 0$, truth of the first premise similarly implies that some literal of C_1 is true, so again $v(C_1 \cup C_2) = 1$. □

Theorem 2.38 (Soundness)**.** *If $\mathcal{C} \vdash_R C$, then every truth assignment satisfying $\mathcal{C}$ satisfies C. Thus every resolution conclusion is a logical consequence of its assumptions.*

Proof. Let $C_0, \ldots, C_m = C$ be a resolution proof from $\mathcal{C}$, and let $v(\mathcal{C}) = 1$. We prove $v(C_i) = 1$ by strong induction on i. For $i = 0$, there are no earlier lines, so $C_0 \in \mathcal{C}$ and $v(C_0) = 1$. Suppose $i > 0$ and $v(C_j) = 1$ for every $j < i$. If $C_i \in \mathcal{C}$, it is true by assumption. Otherwise it is a resolvent of C_k, C_l with $k, l < i$. Both premises are true by the induction hypothesis, and the soundness lemma gives $v(C_i) = 1$. In particular, $v(C_m) = v(C) = 1$. □

Remark 2.39 (Completeness versus refutation completeness). Resolution does not derive every logical consequence: from $\{\{p_0\}\}$ one cannot derive $\{p_0, p_1\}$, although $p_0 \Rightarrow (p_0 \vee p_1)$. Indeed, the only available clause is $\{p_0\}$, and no resolution step is possible without a complementary literal. Resolution is nevertheless *refutation complete*: an unsatisfiable clause set admits a derivation of the empty clause.

2.5.4. *Proving an implication by refutation.* To prove $A \Rightarrow B$, it suffices to show that $A \wedge \neg B$ is unsatisfiable: a truth assignment falsifying the implication would make both A and $\neg B$ true. Convert $A \wedge \neg B$ into CNF, encode it as a clause set, and derive $\emptyset$.

Example 2.40. We prove

$$(p_0 \wedge (p_0 \to p_1)) \Rightarrow (p_1 \vee p_2).$$

Negating the conclusion gives $\neg(p_1 \vee p_2) \Leftrightarrow (\neg p_1 \wedge \neg p_2)$. Hence the premises together with the negated conclusion give

$$\mathcal{C} = \{\{p_0\}, \{\neg p_0, p_1\}, \{\neg p_1\}, \{\neg p_2\}\}.$$

A resolution refutation is

0.	$\{p_0\}$	assumption,
1.	$\{\neg p_0, p_1\}$	assumption,
2.	$\{\neg p_1\}$	assumption,
3.	$\{\neg p_2\}$	assumption,
4.	$\{p_1\}$	resolution of lines 0 and 1,
5.	$\emptyset$	resolution of lines 4 and 2.

Thus $\mathcal{C} \vdash_R \emptyset$, so soundness implies that $\mathcal{C}$ is unsatisfiable and proves the implication. Line 3 need not be used: the premises already imply p_1.

2.5.5. *Refutation completeness.*

Theorem 2.41 (Finite refutation completeness)**.** *If $\mathcal{C}$ is a finite unsatisfiable clause set, then $\mathcal{C} \vdash_R \emptyset$.*

Proof. We use strong induction on the number n of distinct propositional symbols occurring in $\mathcal{C}$. This number is finite because both $\mathcal{C}$ and each of its clauses are finite.

If $n = 0$, the only possible clause is $\emptyset$. Since the empty clause set is satisfiable, unsatisfiability forces $\emptyset \in \mathcal{C}$. The one-line proof $(\emptyset)$ suffices.

Suppose $n > 0$ and the result holds for fewer than n symbols. Choose a symbol p_i occurring in $\mathcal{C}$, and partition the clause set as in the book:

$$\mathcal{C}_{00} = \{C \in \mathcal{C} : p_i \notin C,\ \neg p_i \notin C\},$$
$$\mathcal{C}_{01} = \{C \in \mathcal{C} : p_i \notin C,\ \neg p_i \in C\},$$
$$\mathcal{C}_{10} = \{C \in \mathcal{C} : p_i \in C,\ \neg p_i \notin C\},$$
$$\mathcal{C}_{11} = \{C \in \mathcal{C} : p_i \in C,\ \neg p_i \in C\}.$$

Every clause in $\mathcal{C}_{11}$ is tautological. Form all resolvents between the negative and positive parts on the pivot p_i:

$$\mathcal{C}_{01} \times_R \mathcal{C}_{10} = \{(C_1 \setminus \{\neg p_i\}) \cup (C_2 \setminus \{p_i\}) : C_1 \in \mathcal{C}_{01},\ C_2 \in \mathcal{C}_{10}\}.$$

Define

$$\mathcal{C}' = \mathcal{C}_{00} \cup (\mathcal{C}_{01} \times_R \mathcal{C}_{10}).$$

No clause of $\mathcal{C}'$ contains p_i or $\neg p_i$, so $\mathcal{C}'$ contains at most $n - 1$ distinct symbols.

Claim: $\mathcal{C}'$ is unsatisfiable. Suppose instead that $v(\mathcal{C}') = 1$. We show that changing only the value of p_i, if necessary, gives a satisfying assignment for $\mathcal{C}$. There are two cases.

(1) Suppose $v(C \setminus \{\neg p_i\}) = 1$ for every $C \in \mathcal{C}_{01}$. Define $w(p_i) = 1$ and $w(p_j) = v(p_j)$ for $j \neq i$. Each clause of $\mathcal{C}_{01}$ remains true because its part not involving p_i is true. Each clause of $\mathcal{C}_{10}$ is true because it contains p_i.

(2) Otherwise, choose $C_1 \in \mathcal{C}_{01}$ such that $v(C_1 \setminus \{\neg p_i\}) = 0$. For every $C_2 \in \mathcal{C}_{10}$, the resolvent

$$(C_1 \setminus \{\neg p_i\}) \cup (C_2 \setminus \{p_i\})$$

belongs to $\mathcal{C}'$ and is true under v. Its first part is false, so $v(C_2 \setminus \{p_i\}) = 1$. Define $w(p_i) = 0$ and $w(p_j) = v(p_j)$ for $j \neq i$. Every clause of $\mathcal{C}_{01}$ is true because it contains $\neg p_i$; every clause of $\mathcal{C}_{10}$ is true because its remaining part is true.

In both cases, the clauses of $\mathcal{C}_{00}$ remain true because their truth values do not depend on p_i, and all clauses of $\mathcal{C}_{11}$ are true under every assignment. Thus $w(\mathcal{C}) = 1$, contradicting the unsatisfiability of $\mathcal{C}$. This proves the claim.

By the induction hypothesis, $\mathcal{C}' \vdash_R \emptyset$. To obtain a proof from $\mathcal{C}$, first list the clauses of $\mathcal{C}$ as assumptions and derive all clauses in $\mathcal{C}_{01} \times_R \mathcal{C}_{10}$ by resolution. Every assumption used in the refutation from $\mathcal{C}'$ is now available: it is either in $\mathcal{C}_{00} \subseteq \mathcal{C}$ or among these resolvents. Reproduce the resolution steps of that refutation, referring to the already available lines whenever it uses an assumption. This yields a finite resolution proof of $\emptyset$ from $\mathcal{C}$. □

Remark 2.42 (Scope of the proof). The course book states Lemma 6.7 for arbitrary clause sets, but the induction above proves the finite case treated in the lecture. The general case follows using propositional compactness: every unsatisfiable clause set has a finite unsatisfiable subset, whose refutation is also a refutation from the original set. Compactness is an additional result; it is not proved by this induction.

Corollary 2.43 (Refutation criterion)**.** *For a finite clause set $\mathcal{C}$,*

$$\mathcal{C} \text{ is unsatisfiable} \quad \Longleftrightarrow \quad \mathcal{C} \vdash_R \emptyset.$$

Proof. The forward direction is refutation completeness. Conversely, if $\mathcal{C} \vdash_R \emptyset$, soundness says that any assignment satisfying $\mathcal{C}$ must satisfy $\emptyset$. Since the empty clause is always false, no such assignment exists. □

2.6. **28.09.2026.**

2.6.1. *Natural deduction and assumptions.* Natural deduction gives introduction and elimination rules for each logical connective. An introduction rule tells us how to prove a formula with that connective; an elimination rule tells us what we can infer from such a formula. Each rule has one to three premises and exactly one conclusion. Proofs are drawn as trees, with premises above a horizontal line and the conclusion below it.

Definition 2.44 (Derivability)**.** Let Γ be a set of propositional formulas. We write

$$\Gamma \vdash A$$

if there is a natural deduction proof of A whose undischarged assumptions all belong to Γ. We write $\vdash A$ for $\emptyset \vdash A$: a proof with no undischarged assumptions.

An assumption may be used more than once. Some rules introduce a temporary assumption and later *discharge* it. We mark the relevant occurrences as $[A]^i$ and put the same label i beside the rule that discharges them. Below that rule, these occurrences are no longer open assumptions. Other assumptions remain open. The notation

$$\begin{array}{c} [A]^i \\ \vdots \\ B \end{array}$$

stands for a derivation of B under the temporary assumption A, possibly together with other open assumptions. A derivation may consist of just an assumption, with no inference steps.

We retain the lecture's labels: T means introduction (Finnish *tuonti*) and E means elimination.

2.6.2. *Conjunction.*

Introduction. From proofs of both conjuncts, infer their conjunction:

$$\frac{A \quad B}{A \wedge B}\ \wedge\mathrm{T}.$$

Elimination. From a conjunction, infer either conjunct:

$$\frac{A \wedge B}{A}\ \wedge\mathrm{E} \qquad \frac{A \wedge B}{B}\ \wedge\mathrm{E}.$$

Example 2.45 (Commutativity of conjunction)**.** We prove $\{A \wedge B\} \vdash B \wedge A$:

$$\dfrac{\dfrac{A \wedge B}{B}\ \wedge\mathrm{E} \quad \dfrac{A \wedge B}{A}\ \wedge\mathrm{E}}{B \wedge A}\ \wedge\mathrm{T}$$

Both leaves use the same open assumption $A \wedge B$.

2.6.3. *Disjunction introduction.* From either disjunct, infer the disjunction:

$$\frac{A}{A \vee B}\ \vee\mathrm{T} \qquad \frac{B}{A \vee B}\ \vee\mathrm{T}.$$

Example 2.46 (From a conjunction to a disjunction)**.** There are two proofs of $\{A \wedge B\} \vdash A \vee B$:

$$\dfrac{\dfrac{A \wedge B}{A}\ \wedge\mathrm{E}}{A \vee B}\ \vee\mathrm{T} \quad \text{or} \quad \dfrac{\dfrac{A \wedge B}{B}\ \wedge\mathrm{E}}{A \vee B}\ \vee\mathrm{T}.$$

2.6.4. *Implication elimination.* Implication elimination is *modus ponens*: from $A \to B$ and A, infer B.

$$\frac{A \to B \quad A}{B}\ \to\mathrm{E}.$$

Example 2.47 (Two applications of modus ponens)**.** We prove $\{A \wedge B,\ A \to (B \to C)\} \vdash C$:

$$\dfrac{\dfrac{A \to (B \to C) \qquad \dfrac{A \wedge B}{A}\ \wedge\mathrm{E}}{B \to C}\ \to\mathrm{E} \qquad \dfrac{A \wedge B}{B}\ \wedge\mathrm{E}}{C}\ \to\mathrm{E}$$

First extract A and B from the conjunction. Apply modus ponens with A to obtain $B \to C$, and then with B to obtain C.

2.6.5. *Equivalence and negation elimination.*
Equivalence elimination. An equivalence can be used in either direction:

$$\frac{A \leftrightarrow B \quad A}{B}\ \leftrightarrow\mathrm{E} \qquad \frac{A \leftrightarrow B \quad B}{A}\ \leftrightarrow\mathrm{E}.$$

Negation elimination. The lecture uses double negation elimination:

$$\frac{\neg\neg A}{A}\ \neg\mathrm{E}.$$

This is a classical rule: a proof of $\neg\neg A$ allows us to conclude A.

2.6.6. *Disjunction elimination: proof by cases.* Suppose that $A \vee B$ has been proved. If we can derive the same conclusion C under the assumption A and under the assumption B, we may conclude C and discharge the two case assumptions:

$$\dfrac{A \vee B \qquad \begin{matrix}[A]^i \\ \vdots \\ C\end{matrix} \qquad \begin{matrix}[B]^j \\ \vdots \\ C\end{matrix}}{C}\ \vee\mathrm{E},\ i, j.$$

The two cases are separate subproofs. An assumption made only in one case cannot be used in the other case. Any other open assumptions in the subproofs remain open after this rule.

Example 2.48 (A common conjunct in both cases)**.** We prove $\{(A \wedge C) \vee (B \wedge C)\} \vdash C$:

$$\dfrac{(A \wedge C) \vee (B \wedge C) \quad \dfrac{[A \wedge C]^1}{C} \wedge\text{E} \quad \dfrac{[B \wedge C]^2}{C} \wedge\text{E}}{C} \vee\text{E},\ 1, 2$$

In either case, conjunction elimination gives C. The case assumptions are discharged; the original disjunction remains open.

Example 2.49 (Commutativity of disjunction)**.** We prove $\{A \vee B\} \vdash B \vee A$:

$$\dfrac{A \vee B \quad \dfrac{[A]^1}{B \vee A} \vee\text{T} \quad \dfrac{[B]^2}{B \vee A} \vee\text{T}}{B \vee A} \vee\text{E},\ 1, 2$$

Example 2.50 (Repeated disjuncts)**.** We prove $\{A \vee A\} \vdash A$:

$$\dfrac{A \vee A \quad [A]^1 \quad [A]^2}{A} \vee\text{E},\ 1, 2$$

Each case already has A as its assumption, so no inference is needed within either case.

2.6.7. *Implication introduction.* To prove $A \to B$, assume A, derive B, and discharge the assumption A:

$$\dfrac{\begin{array}{c}[A]^i \\ \vdots \\ B\end{array}}{A \to B} \to\text{T},\ i.$$

The derivation of B may also use other assumptions; these remain open. It is permitted that the temporary assumption is never used, in which case the discharge is called *vacuous*.

Example 2.51 (Introducing a disjunct in the consequent)**.** We prove $\emptyset \vdash A \to (A \vee B)$:

$$\dfrac{\dfrac{[A]^1}{A \vee B} \vee\text{T}}{A \to (A \vee B)} \to\text{T},\ 1$$

Discharging A leaves no open assumptions.

Example 2.52 (An unused temporary assumption)**.** We prove $\{A\} \vdash B \to A$:

$$\dfrac{A}{B \to A} \to\text{T}$$

The conclusion A is already available as an open assumption, so the temporary assumption B need not be used. Its discharge is vacuous; the assumption A remains open.

Example 2.53 (Identity)**.** We prove $\vdash A \to A$:

$$\dfrac{[A]^1}{A \to A} \to\text{T},\ 1$$

The one-formula derivation of A from itself suffices. Discharging it leaves no open assumptions.

2.6.8. *Equivalence introduction.* To prove $A \leftrightarrow B$, derive B assuming A, and derive A assuming B. Discharge both temporary assumptions:

$$\dfrac{\begin{array}{cc}[A]^i & [B]^j \\ \vdots & \vdots \\ B & A\end{array}}{A \leftrightarrow B}\ \leftrightarrow \mathrm{T},\, i, j.$$

Example 2.54 (Additional example: equivalence introduction)**.** We prove $\vdash (A \wedge B) \leftrightarrow (B \wedge A)$. Each direction uses conjunction elimination followed by introduction:

$$\dfrac{\dfrac{\dfrac{[A \wedge B]^1}{B}\wedge\mathrm{E} \quad \dfrac{[A \wedge B]^1}{A}\wedge\mathrm{E}}{B \wedge A}\wedge\mathrm{T} \qquad \dfrac{\dfrac{[B \wedge A]^2}{A}\wedge\mathrm{E} \quad \dfrac{[B \wedge A]^2}{B}\wedge\mathrm{E}}{A \wedge B}\wedge\mathrm{T}}{(A \wedge B) \leftrightarrow (B \wedge A)}\ \leftrightarrow \mathrm{T},\, 1, 2$$

The final rule discharges both occurrences labelled 1 and both occurrences labelled 2. There are no open assumptions.

2.6.9. *Negation introduction.* To prove $\neg A$, assume A and derive a contradiction $B \wedge \neg B$. Then discharge the assumption A:

$$\dfrac{\begin{array}{c}[A]^i \\ \vdots \\ B \wedge \neg B\end{array}}{\neg A}\ \neg\mathrm{T},\, i.$$

The formula B can be any formula; it need not be the same as A.

Example 2.55 (Additional example: negation introduction (modus tollens))**.** We prove $\{A \to B,\ \neg B\} \vdash \neg A$. Assume A. Modus ponens gives B, which contradicts the premise $\neg B$:

$$\dfrac{\dfrac{\dfrac{A \to B \quad [A]^1}{B}\to\mathrm{E} \qquad \neg B}{B \wedge \neg B}\wedge\mathrm{T}}{\neg A}\ \neg\mathrm{T},\, 1$$

Only A is discharged. The premises $A \to B$ and $\neg B$ remain open.

2.7. **01.10.2026.**

2.7.1. *Planning a natural deduction proof.* Two useful questions are: *What may I assume?* and *What do I want to derive?* Work backwards from the main connective of the goal. For an implication, assume its antecedent and aim for its consequent; for a negation, assume the formula to be negated and aim for a contradiction. Keep track of which rule will discharge each temporary assumption.

Example 2.56 (Introducing an implication)**.** To prove $\vdash A \to (A \vee B)$, we may temporarily assume A. The new goal is $A \vee B$, obtained by disjunction introduction:

$$\dfrac{\dfrac{[A]^1}{A \vee B}\vee\mathrm{T}}{A \to (A \vee B)}\to\mathrm{T},\, 1$$

Example 2.57 (An implication with an unused antecedent)**.** We prove $\{A \to (B \wedge C),\ A \wedge D\} \vdash E \to B$. Besides the premises, we may assume E temporarily. Our goal is B, which

follows from the premises alone:

$$\dfrac{\dfrac{A \to (B \wedge C) \qquad \dfrac{A \wedge D}{A}\,\wedge\mathrm{E}}{\dfrac{B \wedge C}{B}\,\wedge\mathrm{E}}\,\to\mathrm{E}}{E \to B}\,\to\mathrm{T}$$

The discharge of E is vacuous, since it was never used.

Example 2.58 (Commutativity of conjunction)**.** For an equivalence, prove each direction under its own assumption:

$$\dfrac{\dfrac{\dfrac{[A \wedge B]^1}{B}\,\wedge\mathrm{E} \quad \dfrac{[A \wedge B]^1}{A}\,\wedge\mathrm{E}}{B \wedge A}\,\wedge\mathrm{T} \qquad \dfrac{\dfrac{[B \wedge A]^2}{A}\,\wedge\mathrm{E} \quad \dfrac{[B \wedge A]^2}{B}\,\wedge\mathrm{E}}{A \wedge B}\,\wedge\mathrm{T}}{(A \wedge B) \leftrightarrow (B \wedge A)}\,\leftrightarrow\mathrm{T},\,1,2$$

All four assumption occurrences are discharged by the final rule.

2.7.2. *De Morgan's law for a disjunction.*

Example 2.59 (From two negations to the negation of a disjunction)**.** We prove $\vdash (\neg A \wedge \neg B) \to \neg(A \vee B)$. Assume $\neg A \wedge \neg B$, and then assume $A \vee B$ to derive a contradiction. In each case of the disjunction, we can derive $\neg(A \vee B)$. Write these two subproofs as

$$\mathcal{P}_A = \dfrac{\dfrac{[A]^1 \qquad \dfrac{[\neg A \wedge \neg B]^3}{\neg A}\,\wedge\mathrm{E}}{A \wedge \neg A}\,\wedge\mathrm{T}}{\neg(A \vee B)}\,\neg\mathrm{T}\ \text{(vacuous)}$$

and

$$\mathcal{P}_B = \dfrac{\dfrac{[B]^1 \qquad \dfrac{[\neg A \wedge \neg B]^3}{\neg B}\,\wedge\mathrm{E}}{B \wedge \neg B}\,\wedge\mathrm{T}}{\neg(A \vee B)}\,\neg\mathrm{T}\ \text{(vacuous)}.$$

The negation introductions here discharge no assumptions: each contradiction already follows from the open assumptions in that case. In particular, the case assumptions labelled 1 remain open until disjunction elimination. Combining the subproofs gives

$$\dfrac{\dfrac{\dfrac{[A \vee B]^2 \qquad \dfrac{[A \vee B]^2 \quad \mathcal{P}_A \quad \mathcal{P}_B}{\neg(A \vee B)}\,\vee\mathrm{E},\,1}{(A \vee B) \wedge \neg(A \vee B)}\,\wedge\mathrm{T}}{\neg(A \vee B)}\,\neg\mathrm{T},\,2}{(\neg A \wedge \neg B) \to \neg(A \vee B)}\,\to\mathrm{T},\,3$$

Alternative proof with a common contradiction. Disjunction elimination requires the same conclusion in both cases. We can use $B \wedge \neg B$ as that conclusion. In the A-case, first obtain $A \wedge \neg A$, and use a vacuous negation introduction followed by double negation elimination:

$$\mathcal{R}_A = \dfrac{\dfrac{\dfrac{[A]^1 \qquad \dfrac{[\neg A \wedge \neg B]^3}{\neg A}\,\wedge\mathrm{E}}{A \wedge \neg A}\,\wedge\mathrm{T}}{\neg\neg(B \wedge \neg B)}\,\neg\mathrm{T}\ \text{(vacuous)}}{B \wedge \neg B}\,\neg\mathrm{E}\qquad.$$

In the B-case, the contradiction is immediate:

$$\mathcal{R}_B = \dfrac{[B]^1 \quad \dfrac{[\neg A \wedge \neg B]^3}{\neg B}\,\wedge\mathrm{E}}{B \wedge \neg B}\,\wedge\mathrm{T}.$$

Now finish with

$$\dfrac{\dfrac{[A \vee B]^2 \quad \mathcal{R}_A \quad \mathcal{R}_B}{\dfrac{B \wedge \neg B}{\neg(A \vee B)}\,\neg\mathrm{T},\ 2}\,\vee\mathrm{E},\ 1}{(\neg A \wedge \neg B) \to \neg(A \vee B)}\,\to\mathrm{T},\ 3.$$

Remark 2.60 (Corrections to the handwritten proof). The common conclusion of the case split is $B \wedge \neg B$, rather than $B \vee \neg B$. Both branches must yield a contradiction for the subsequent negation introduction.

Example 2.61 (From the negation of a disjunction to two negations)**.** We prove $\vdash \neg(A \vee B) \to (\neg A \wedge \neg B)$. Assume $\neg(A \vee B)$. To obtain the conjunction, prove $\neg A$ and $\neg B$ separately, using temporary assumptions A and B, respectively:

$$\mathcal{P} = \dfrac{\dfrac{\dfrac{[A]^1}{A \vee B}\,\vee\mathrm{T} \quad [\neg(A \vee B)]^3}{(A \vee B) \wedge \neg(A \vee B)}\,\wedge\mathrm{T}}{\neg A}\,\neg\mathrm{T},\ 1,$$

$$\mathcal{Q} = \dfrac{\dfrac{\dfrac{[B]^2}{A \vee B}\,\vee\mathrm{T} \quad [\neg(A \vee B)]^3}{(A \vee B) \wedge \neg(A \vee B)}\,\wedge\mathrm{T}}{\neg B}\,\neg\mathrm{T},\ 2.$$

Combine these subproofs and discharge the original assumption:

$$\dfrac{\dfrac{\mathcal{P} \quad \mathcal{Q}}{\neg A \wedge \neg B}\,\wedge\mathrm{T}}{\neg(A \vee B) \to (\neg A \wedge \neg B)}\,\to\mathrm{T},\ 3$$

2.7.3. *De Morgan's law for a conjunction.*

Example 2.62 (From a disjunction of negations to a negated conjunction)**.** We prove $\vdash (\neg A \vee \neg B) \to \neg(A \wedge B)$. Assume $\neg A \vee \neg B$, and temporarily assume $A \wedge B$. Split into the cases $\neg A$ and $\neg B$. The two subproofs are

$$\mathcal{P} = \dfrac{\dfrac{\dfrac{[A \wedge B]^2}{A}\,\wedge\mathrm{E} \quad [\neg A]^3}{A \wedge \neg A}\,\wedge\mathrm{T}}{\neg(A \wedge B)}\,\neg\mathrm{T}\ (\text{vacuous}),$$

$$\mathcal{Q} = \dfrac{\dfrac{\dfrac{[A \wedge B]^2}{B}\,\wedge\mathrm{E} \quad [\neg B]^3}{B \wedge \neg B}\,\wedge\mathrm{T}}{\neg(A \wedge B)}\,\neg\mathrm{T}\ (\text{vacuous}).$$

The case assumptions labelled 3 are discharged by $\vee$E. The assumption labelled 2 remains open through both subproofs and is discharged after the cases have been combined:

$$\dfrac{\dfrac{\dfrac{[\neg A \vee \neg B]^{1} \quad \mathcal{P} \quad \mathcal{Q}}{\neg(A \wedge B)} \vee\text{E, } 3 \qquad [A \wedge B]^{2}}{(A \wedge B) \wedge \neg(A \wedge B)} \wedge\text{T}}{\dfrac{\neg(A \wedge B)}{(\neg A \vee \neg B) \to \neg(A \wedge B)} \to\text{T, } 1} \neg\text{T, } 2$$

Remark 2.63 (Discharge labels). In the handwritten tree, the negation introductions inside the cases are marked with the case label 3. Those steps are vacuous: the assumptions $\neg A$ and $\neg B$ must instead remain open until the case split is closed by disjunction elimination.

Example 2.64 (From a negated conjunction to a disjunction of negations)**.** We prove $\vdash \neg(A \wedge B) \to (\neg A \vee \neg B)$. Assume $\neg(A \wedge B)$. To derive the disjunction, assume its negation $\neg(\neg A \vee \neg B)$, derive a contradiction, and apply double negation elimination.

First assume B, and within that subproof assume A. The conjunction $A \wedge B$ contradicts the original assumption, allowing us to discharge A:

$$\mathcal{P} = \dfrac{\dfrac{\dfrac{[A]^{1} \quad [B]^{2}}{A \wedge B} \wedge\text{T} \qquad [\neg(A \wedge B)]^{4}}{(A \wedge B) \wedge \neg(A \wedge B)} \wedge\text{T}}{\neg A} \neg\text{T, } 1 .$$

From $\neg A$, introduce $\neg A \vee \neg B$. This contradicts the assumption labelled 3, so we can discharge B:

$$\mathcal{Q} = \dfrac{\dfrac{\dfrac{\mathcal{P}}{\neg A \vee \neg B} \vee\text{T} \qquad [\neg(\neg A \vee \neg B)]^{3}}{(\neg A \vee \neg B) \wedge \neg(\neg A \vee \neg B)} \wedge\text{T}}{\neg B} \neg\text{T, } 2 .$$

Introduce the disjunction again, now from $\neg B$, to obtain the final contradiction:

$$\dfrac{\dfrac{\dfrac{\dfrac{\mathcal{Q}}{\neg A \vee \neg B} \vee\text{T} \qquad [\neg(\neg A \vee \neg B)]^{3}}{(\neg A \vee \neg B) \wedge \neg(\neg A \vee \neg B)} \wedge\text{T}}{\neg\neg(\neg A \vee \neg B)} \neg\text{T, } 3}{\neg A \vee \neg B} \neg\text{E}}{\neg(A \wedge B) \to (\neg A \vee \neg B)} \to\text{T, } 4$$

All assumptions have now been discharged. This proof uses the classical double negation elimination rule.

2.8. **05.10.2026.**

2.8.1. *Derivations and their open assumptions.* The inference rules above define natural deduction derivations inductively. For a derivation $\mathcal{D}$, let $O(\mathcal{D})$ be its set of *open assumption occurrences*. We track occurrences rather than just formulas, since a rule may discharge some occurrences of A while leaving other occurrences of the same formula open.

Definition 2.65 (Natural deduction derivation)**.** A derivation is a finite proof tree obtained by the following clauses.

(1) For every formula A, the one-node tree A is a derivation with conclusion A. Its sole occurrence of A is open.

(2) If derivations have the conclusions required as premises of any natural deduction rule listed above, applying that rule gives a new derivation with the conclusion displayed by the rule.

(3) The open assumptions of the new derivation are the open assumptions of its premise derivations, except for the occurrences explicitly discharged by that rule. No other trees are derivations.

In particular, rules that discharge no assumptions take the union of the open assumptions of their premise derivations. If $\mathcal{P}$ concludes A and $\mathcal{Q}$ concludes B, then

$$O\left(\dfrac{\begin{matrix}\mathcal{P} & \mathcal{Q}\end{matrix}}{A \wedge B}\ \wedge\mathrm{T}\right) = O(\mathcal{P}) \cup O(\mathcal{Q}).$$

If $\mathcal{P}$ concludes $A \wedge B$, either conjunction elimination retains exactly $O(\mathcal{P})$. The same union rule applies to disjunction introduction, implication and equivalence elimination, and double negation elimination.

For a rule that discharges assumptions, let S_A denote the selected open occurrences of A in the relevant subderivation. For example, implication introduction changes $O(\mathcal{Q})$ to $O(\mathcal{Q}) \setminus S_A$, and negation introduction does the same. Equivalence introduction removes its selected A-occurrences from the first subderivation and its selected B-occurrences from the second. For disjunction elimination, where $\mathcal{P}$ proves $A \vee B$, $\mathcal{Q}$ proves C under A, and $\mathcal{R}$ proves C under B, the open assumptions are

$$O(\mathcal{P}) \cup (O(\mathcal{Q}) \setminus S_A) \cup (O(\mathcal{R}) \setminus S_B).$$

Here S_A and S_B contain only the case assumptions selected for discharge. These sets may be empty when a discharge is vacuous.

2.8.2. *Soundness of natural deduction.* For a truth assignment v, write $v \models \Gamma$ if $v(G) = 1$ for every $G \in \Gamma$. We write $\Gamma \models A$ if every assignment satisfying Γ also satisfies A.

Theorem 2.66 (Soundness of natural deduction)**.** *If $\Gamma \vdash A$, then $\Gamma \models A$. More precisely, if a derivation $\mathcal{D}$ concludes A, then every truth assignment making all assumptions in $O(\mathcal{D})$ true also makes A true.*

Proof. We prove the stronger statement by induction on the construction of $\mathcal{D}$. If $\mathcal{D}$ is the assumption leaf A, the claim is immediate.

Suppose the last step applies a rule to smaller derivations and the induction hypothesis holds for each of them. Fix an assignment v satisfying all open assumptions of the resulting derivation. For rules with no discharge, v satisfies the open assumptions of every premise derivation. The induction hypothesis therefore makes their conclusions true. The truth tables verify each inference:

- $\wedge$T combines two true formulas, and $\wedge$E extracts a true conjunct.
- $\vee$T makes a disjunction true from either true disjunct.
- $\to$ E uses $v(A \to B) = v(A) = 1$ to obtain $v(B) = 1$.
- Either $\leftrightarrow$ E rule uses the equal truth values of A and B.
- $\neg$E uses $v(\neg\neg A) = 1$, which implies $v(A) = 1$.

It remains to check the rules that discharge assumptions.

- For $\to$ T, let the premise derivation prove B while some occurrences of A are discharged. If $v(A) = 0$, then $v(A \to B) = 1$. If $v(A) = 1$, v satisfies the discharged occurrences too. The induction hypothesis gives $v(B) = 1$, so again $v(A \to B) = 1$.
- For $\leftrightarrow$ T, the first subderivation proves B under discharged assumptions A, and the second proves A under discharged assumptions B. By the induction hypothesis, $v(A) = 1$ implies $v(B) = 1$, and $v(B) = 1$ implies $v(A) = 1$. Thus $v(A \leftrightarrow B) = 1$.
- For $\neg$T, the premise derivation proves $B \wedge \neg B$ while some occurrences of A are discharged. If $v(A) = 1$, the induction hypothesis would make $v(B \wedge \neg B) = 1$, which is impossible. Hence $v(A) = 0$ and $v(\neg A) = 1$.

- For $\vee$E, the major premise proves $A \vee B$, so its induction hypothesis gives $v(A \vee B) = 1$. If $v(A) = 1$, the induction hypothesis for the A-case yields $v(C) = 1$: its discharged occurrences of A are true, and its other open assumptions are true by choice of v. If $v(A) = 0$, then $v(B) = 1$, and the same argument with the B-case yields $v(C) = 1$.

This proves the stronger statement for every derivation. If $\Gamma \vdash A$, the open assumptions of a witnessing derivation all belong to Γ, so every model of Γ makes A true. $\square$

Corollary 2.67 (Using soundness to rule out a derivation)**.** *If* $\Gamma \not\models A$*, then* $\Gamma \nvdash A$*.*

Example 2.68. The assignment $v(p_0) = 0$ and $v(p_1) = 1$ satisfies both p_1 and $p_0 \to p_1$, but it falsifies $p_0 \wedge p_1$. Hence

$$\{p_1, p_0 \to p_1\} \not\models p_0 \wedge p_1, \qquad \text{and therefore} \qquad \{p_1, p_0 \to p_1\} \nvdash p_0 \wedge p_1.$$

Example 2.69 (A consequence of a symmetric premise)**.** Can we infer $(p_0 \to p_1) \to p_1$ from $(p_1 \to p_0) \to p_0$? The two formulas are logically equivalent:

$$(p_0 \to p_1) \to p_1 \Leftrightarrow p_0 \vee p_1 \Leftrightarrow (p_1 \to p_0) \to p_0.$$

Indeed, $(X \to Y) \to Y$ has truth value 1 exactly when $X \vee Y$ does. Thus the semantic inference is valid:

$$\{(p_1 \to p_0) \to p_0\} \models (p_0 \to p_1) \to p_1.$$

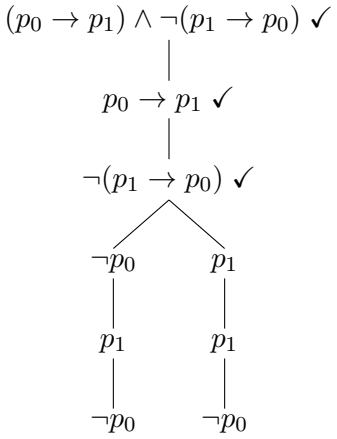
2.9. **08.10.2026.**

2.9.1. *Counterexamples and semantic trees.*

Example 2.70 (An implication does not imply its converse)**.** To show that

$$\{p_0 \to p_1\} \not\models p_1 \to p_0,$$

we seek a truth assignment satisfying $(p_0 \to p_1) \wedge \neg(p_1 \to p_0)$. The following tree decomposes the requirements for this formula to be true:



Both branches give the same counterexample: $v(p_0) = 0$ and $v(p_1) = 1$. The premise is true, whereas its converse is false. By soundness, the converse is not derivable from the premise either.

A *semantic tree* (or *semantic tableau*) is a method for finding a truth assignment satisfying a formula, or showing that no such assignment exists. It can require less work than writing out a full truth table. The idea is to decompose truth requirements: write formulas on the same branch when both are required, and split into branches when there are alternative ways to satisfy a formula.

Definition 2.71 (Open, closed, and completed branches)**.** Mark a formula with ✓ after applying its expansion rule. A branch containing both A and $\neg A$ is *closed*, marked $\times$. A branch that is not closed is *open*. A tree is *closed* when all its branches are closed.

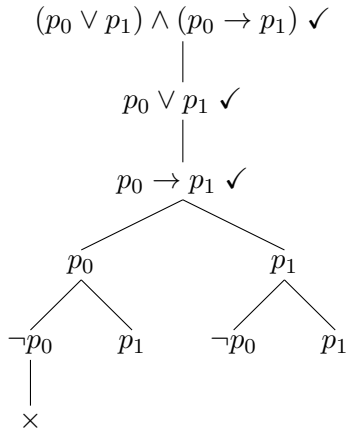
An open branch is *completed* when all its nonliteral formulas have been expanded, so that only literals remain unmarked. A satisfying truth assignment can then be read from it: set $v(p_n) = 1$ if p_n occurs on the branch and $v(p_n) = 0$ if $\neg p_n$ occurs. Symbols absent from the branch may be assigned either value.

2.9.2. *Expansion rules.* Each formula on a branch is required to be true. In the table below, commas mean that both formulas are added to the same branch; the vertical bar separates two alternative branches. Apply a rule to every open branch below the formula being expanded.

Formula	Expansion	Type
$A \wedge B$	$A,\ B$	Same branch
$\neg(A \wedge B)$	$\neg A \mid \neg B$	Split
$A \vee B$	$A \mid B$	Split
$\neg(A \vee B)$	$\neg A,\ \neg B$	Same branch
$A \to B$	$\neg A \mid B$	Split
$\neg(A \to B)$	$A,\ \neg B$	Same branch
$A \leftrightarrow B$	$(A,\ B) \mid (\neg A,\ \neg B)$	Split
$\neg(A \leftrightarrow B)$	$(A,\ \neg B) \mid (\neg A,\ B)$	Split
$\neg\neg A$	A	Same branch

For example, $\neg(A \to B)$ requires both A and $\neg B$ on the same branch. A branch closes as soon as it contains a contradictory pair, in either order; the pair need not consist of adjacent formulas or of literals. There is no need to expand a closed branch further.

Example 2.72 (Finding a satisfying assignment)**.** Consider $(p_0 \vee p_1) \wedge (p_0 \to p_1)$. Its completed tree is



The leftmost branch closes, but the other branches remain open. For example, the branch containing p_0, p_1 gives the satisfying assignment $v(p_0) = v(p_1) = 1$.

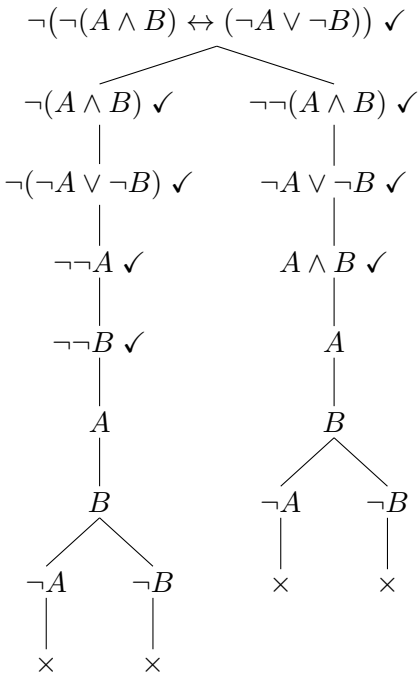
2.9.3. *Semantic proofs.*

Definition 2.73 (Semantic proof)**.** A *semantic proof* of a formula A is a closed semantic tree starting from $\neg A$. It shows that $\neg A$ is unsatisfiable, and hence that A is a tautology.

Example 2.74 (De Morgan's law)**.** To show that

$$\neg(A \wedge B) \Leftrightarrow (\neg A \vee \neg B),$$

construct a tree for the negation of the corresponding equivalence:

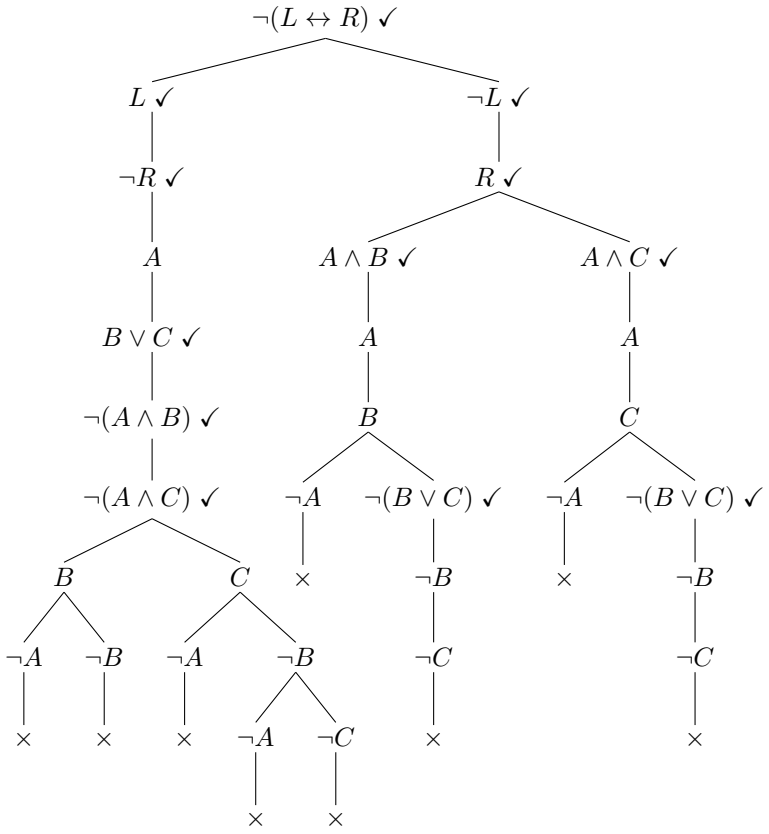


Every branch closes: each contains either $A, \neg A$ or $B, \neg B$. The negated equivalence is unsatisfiable, so the original equivalence is a tautology.

Example 2.75 (Distributivity)**.** We give a semantic proof of

$$(A \wedge (B \vee C)) \leftrightarrow ((A \wedge B) \vee (A \wedge C)).$$

Abbreviate the two sides by $L = A \wedge (B \vee C)$ and $R = (A \wedge B) \vee (A \wedge C)$. Expanding $\neg(L \leftrightarrow R)$ gives the two main branches below. Each formula occupies its own node; vertical lines join formulas required on the same branch.



On the left, L yields A and $B \vee C$, while $\neg R$ yields $\neg(A \wedge B)$ and $\neg(A \wedge C)$. In the B-case, expanding $\neg(A \wedge B)$ closes both branches. In the C-case, the remaining branch closes after expanding $\neg(A \wedge C)$.

On the right, R gives the alternatives $A \wedge B$ and $A \wedge C$. In each case, $\neg L$ gives the alternatives $\neg A$ and $\neg(B \vee C)$. The former contradicts A; the latter yields $\neg B$ and $\neg C$, contradicting B or C, respectively.

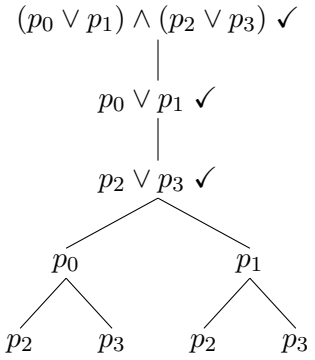
Every branch closes, so the negated equivalence is unsatisfiable and distributivity is a tautology. A checkmark means that a formula has been expanded on every branch where it is still needed; closed branches require no further expansion.

2.9.4. *Satisfiability is not tautologicity.* A completed tree for A tells us whether A is satisfiable. A completed tree for $\neg A$ tells us whether A is falsifiable. These give the following classification:

Classification of A	Tree for A	Tree for $\neg A$
Tautology	Has an open branch	Closed
Contradiction	Closed	Has an open branch
Contingent	Has an open branch	Has an open branch

Thus the satisfiable formulas and the falsifiable formulas overlap exactly in the contingent formulas. Tautologies are satisfiable but not falsifiable; contradictions are falsifiable but not satisfiable.

Example 2.76 (All branches open does not mean tautology)**.** The tree for $(p_0 \vee p_1) \wedge (p_2 \vee p_3)$ has four open branches:



This shows that the formula is satisfiable, but it is not a tautology: setting all four symbols to 0 makes it false. Open branches describe ways to satisfy the formula; they do not assert that every truth assignment satisfies it.

2.10. **12.10.2026.**

2.10.1. *Completeness and syntactic consistency.* The goal is to prove the converse of soundness: if $\Gamma \models A$, then $\Gamma \vdash A$. We prove its contrapositive:

$$\Gamma \nvdash A \quad \Longrightarrow \quad \Gamma \not\models A.$$

Thus we must construct a truth assignment satisfying every formula in Γ but making A false.

Definition 2.77 (Syntactic consistency)**.** A set Γ of propositional formulas is *syntactically inconsistent* if, for some formula B,

$$\Gamma \vdash B \wedge \neg B.$$

Otherwise it is *syntactically consistent*.

Example 2.78. The set $\{p_0, \neg p_0\}$ is syntactically inconsistent, since conjunction introduction gives $\{p_0, \neg p_0\} \vdash p_0 \wedge \neg p_0$.

Semantic consistency means that some truth assignment satisfies every formula in the set. By soundness, syntactic inconsistency implies semantic inconsistency: no truth assignment can satisfy $B \wedge \neg B$. In the following, "consistent" means syntactically consistent.

Lemma 2.79. *For every set S of formulas and every formula A,*

$$S \cup \{\neg A\} \text{ is inconsistent} \quad \Longleftrightarrow \quad S \vdash A.$$

Proof. Suppose first that a derivation $\mathcal{P}$ from $S \cup \{\neg A\}$ concludes $B \wedge \neg B$. Discharge the added assumption $\neg A$ by negation introduction, then apply double negation elimination:

$$\dfrac{\dfrac{\begin{array}{c} S \qquad [\neg A]^1 \\ \mathcal{P} \\ B \wedge \neg B \end{array}}{\neg\neg A}\ \neg\mathrm{T},\ 1}{A}\ \neg\mathrm{E}$$

All remaining open assumptions belong to S, so $S \vdash A$.

Conversely, if $\mathcal{Q}$ derives A from S, then

$$\dfrac{\begin{array}{c} S \\ \mathcal{Q} \qquad \neg A \\ A \end{array}}{A \wedge \neg A}\ \wedge\mathrm{T}$$

is a derivation from $S \cup \{\neg A\}$, proving inconsistency. □

2.10.2. *Maximal consistency and Lindenbaum's lemma.*

Definition 2.80 (Maximal syntactic consistency)**.** A set Γ is *maximally consistent* if it is consistent and every proper superset $\Gamma' \supsetneq \Gamma$ of propositional formulas is inconsistent.

Lemma 2.81 (Lindenbaum's lemma)**.** *Every consistent set of propositional formulas can be extended to a maximally consistent set.*

Proof. Let Γ be consistent and enumerate all propositional formulas as $A_0, A_1, A_2, \ldots$. Define

$$S_0 = \Gamma, \qquad S_{n+1} = \begin{cases} S_n \cup \{A_n\}, & \text{if this set is consistent,} \\ S_n \cup \{\neg A_n\}, & \text{otherwise,} \end{cases}$$

and put $\Gamma' = \bigcup_{n \in \mathbb{N}} S_n$. The sets form an increasing chain, and $\Gamma \subseteq \Gamma'$.

Claim 1: every S_n is consistent. The base case $S_0 = \Gamma$ holds by assumption. Suppose S_n is consistent. If $S_n \cup \{A_n\}$ is consistent, the claim follows by construction. Otherwise we choose $S_{n+1} = S_n \cup \{\neg A_n\}$. If this were inconsistent too, lemma 2.79 would give $S_n \vdash A_n$. Substitute this derivation for the added A_n-assumptions in a derivation of a contradiction from $S_n \cup \{A_n\}$. This yields a contradiction from S_n alone, contrary to the induction hypothesis. Thus S_{n+1} is consistent, and induction proves the claim.

Claim 2: Γ' is consistent. Suppose $\Gamma' \vdash B \wedge \neg B$. A derivation uses only finitely many open assumptions. Since the S_n form an increasing chain, all these assumptions belong to some single S_n. Hence $S_n \vdash B \wedge \neg B$, contradicting Claim 1.

Claim 3: Γ' is maximally consistent. If $A \notin \Gamma'$, then $\neg A \in \Gamma'$ by construction: when A was considered in the enumeration, its negation must have been added. Consequently,

$$\Gamma' \cup \{A\} \vdash A \wedge \neg A.$$

Every proper superset of Γ' contains such an A, so every proper superset is inconsistent. □

2.10.3. *A truth assignment from a maximally consistent set.* Let Γ' be maximally consistent. Define a truth assignment on propositional symbols by

$$v(p_i) = \begin{cases} 1, & \text{if } p_i \in \Gamma', \\ 0, & \text{if } p_i \notin \Gamma'. \end{cases}$$

Extend v to all formulas using the usual truth tables.

Lemma 2.82 (Truth lemma)**.** *For every propositional formula B,*

$$v(B) = 1 \quad \iff \quad B \in \Gamma'.$$

The proof of this lemma was omitted in the lecture. In particular, the lemma says that $v \models \Gamma'$.

2.10.4. *Completeness of natural deduction.*

Theorem 2.83 (Completeness of natural deduction)**.** *If $\Gamma \models A$, then $\Gamma \vdash A$.*

Proof. We prove the contrapositive. Suppose $\Gamma \nvdash A$. By lemma 2.79, the set $\Gamma \cup \{\neg A\}$ is consistent. Lindenbaum's lemma gives a maximally consistent extension

$$\Gamma' \supseteq \Gamma \cup \{\neg A\}.$$

Define $v(p_i) = 1$ exactly when $p_i \in \Gamma'$. By the truth lemma, $v \models \Gamma'$. In particular, $v \models \Gamma$ and $v(\neg A) = 1$, so $v(A) = 0$. This is the required counterexample to semantic consequence: $\Gamma \not\models A$. □